A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

Thai Nguyen Vu[1], Dung Van Bui[2,3], Nhut Tri Do[2,3], and Van Du Nguyen[4]

[1] Campus in Ho Chi Minh City, University of Transport and Communications, Vietnam
thaivn_ph@utc.edu.vn
[2] Vietnam National University Ho Chi Minh City, Vietnam
[3] University of Information Technology, Vietnam
dungbv.17@grad.uit.edu.vn, trinhutdo@uit.edu.vn
[4] Faculty of Information Technology, Nong Lam University, Vietnam
nvdu@hcmuaf.edu.vn

Abstract Deploying machine learning (ML) based intrusion detection on resource-constrained edge devices requires balancing accuracy, latency, and operational cost. We present a split-deployment IDS architecture across two NVIDIA Jetson Orin Nano Super devices connected through Apache Kafka. The system implements a two-stage pipeline: (1) a lightweight autoencoder anomaly gate on Jetson #1 filters benign traffic, and (2) a PySpark `PipelineModel` classifier on Jetson #2 processes only suspicious flows. Central infrastructure (Kafka, PostgreSQL, InfluxDB, Grafana) runs on a host PC. The classifier is the model selected by our companion Spark ML study under a leakage-aware evaluation (label-leaking port features removed, feature-level duplicate flows eliminated). We evaluate three distributed modes (pipeline split, horizontal Kafka consumer scaling, and a Spark standalone cluster) while streaming CICIDS2017 network flows. We report throughput, latency, and the gate's recall–skip trade-off across the three deployment modes: decoupling the gate from the classifier raises verdict throughput from 2.6 to 62.5 flows/s (24×) at a run-level p95 of ≈13 ms, with the gate skipping 95.8% of benign traffic.

Keywords: Intrusion detection · Edge computing · Jetson Orin Nano Super · Kafka · PySpark · Stream processing.

1 Introduction

Network intrusion detection systems (IDS) increasingly must operate closer to data sources to reduce bandwidth and response time [25, 8]. Recent work confirms that accurate detectors now fit on single-board hardware: ensemble and metaheuristic-optimized detectors have been demonstrated on IoT-edge platforms [4, 18], lightweight models have been designed for deployment on industrial control devices [28], and NVIDIA Jetson-class boards have been used both for autoencoder-based IIoT detection [9] and, on the Jetson Orin Nano specifically, for generative-AI-assisted traffic classification [5]. Such devices nonetheless offer limited CPU, memory, and thermal headroom, so a monolithic detector that classifies every flow saturates the board quickly.

Three gaps persist in this literature. First, the recent edge-IDS studies above evaluate a *single* device: streaming middleware such as Kafka has been paired with Spark for distributed detection [17], but the resulting pipelines are not measured on a multi-node cluster of constrained edge boards with explicit role separation, so it remains unclear how much a second board actually buys and in which distribution mode. Second, energy is now a design constraint in its own right at the edge [19], yet throughput, latency percentiles, and per-inference energy are rarely reported together for one deployed system. Third, offline detection scores are increasingly under scrutiny: methodological pitfalls in security ML [2] and documented labeling and duplication errors in CICIDS2017 [13] mean that a model selected under a leaky protocol may be optimistic by several F1 points before it is ever deployed.

We address all three at once. A practical edge IDS must decide *which* model to deploy (small, accurate, explainable, and selected under a leakage-aware protocol) and *how* to distribute inference across nodes under streaming load. We therefore split detection into a lightweight autoencoder gate and a heavier PySpark classifier placed on separate Jetson Orin Nano Super boards, and we measure the resulting system end to end.

Contributions. (1) A two-stage distributed IDS combining a lightweight anomaly gate with a Spark-based classifier; (2) Three deployment modes for a two-node Jetson cluster (pipeline split, horizontal scaling, Spark standalone cluster); (3) An end-to-end performance evaluation reporting throughput, latency percentiles, per-node resource usage, and energy per verdict, metrics that offline studies typically omit; (4) Open, reproducible scripts integrated with a leakage-aware Spark ML training pipeline, so the deployed model is the one validated under a rigorous offline protocol; all code and configurations are publicly available[5].

2 Related Work

Edge and on-device intrusion detection. Edge computing pushes analytics closer to data sources to reduce latency and bandwidth [25, 8], and successive work has shrunk IDS models until they fit single-board hardware. Early systems such as RPiDS [23] showed that a complete detection stack could run on a Raspberry Pi at all; recent studies instead optimize the detector itself, via deep ensemble stacking on IoT-edge platforms [4], metaheuristic hyper-parameter search combined with explainability [18], and compact models tailored to industrial control traffic [28]. NVIDIA Jetson boards are a recurring target: autoencoder-based feature learning reaches sub-millisecond inference on a Jetson Nano [9], and generative models have been run on the Jetson Orin Nano to classify traffic and synthesize training data at once [5]. These studies share a single-device assumption: the detector is benchmarked in isolation on one board, so how detection work should be placed across several boards, and what a second board actually buys, remain open.

Streaming and distributed detection pipelines. Kafka [12] decouples producers from consumers and has become the de-facto ingest layer for streaming analytics, while

[5] `https://github.com/vuthainguyen1602/Thesis_IDS` (branch `develop`).

Spark [27] and its MLlib library [16] supply the distributed training and scoring primitives. Combining the two for intrusion detection is an established pattern; Mohan Kiran Kumar et al. [17], for instance, route Kafka streams into Spark classifiers for distributed detection. Such pipelines are conceived for datacenter-class resources, where an additional executor is cheap, whereas on an 8 GB ARM board the JVM footprint is itself a design constraint, and the corresponding per-node CPU, memory, and energy traces are rarely published. More broadly, ML-based IDS surveys [11, 6, 26] catalog detection accuracy across dozens of models but seldom report the end-to-end throughput or latency percentiles of a running deployment.

Two-stage and split inference. Anomaly-detection surveys [3, 21] systematize the family of detectors that model normal behavior alone; this property is what makes a cascaded design attractive, in which a cheap unsupervised stage filters the bulk of normal data and a costlier supervised stage adjudicates the remainder. Unsupervised deep detectors, deep belief networks [1] and autoencoders [14, 9], are the usual choice for the cheap stage, since they can be calibrated on benign traffic alone. Distributing detection across devices has so far been explored mainly along the federated-learning axis, where nodes collaborate during training while inference stays local [20, 10]. Our split is orthogonal and concerns inference: the two stages are physically separated across boards and joined by a Kafka topic, so gate and classifier scale independently, and no prior work applies a PySpark MLlib classifier in this role on the NVIDIA Jetson Orin Nano Super.[6] Placement matters most for energy: recent TinyML work argues that energy per inference is a first-class edge metric [19], and skipping the heavy stage is the most direct way to lower it.

Evaluation protocol. A parallel line of work questions how faithfully offline scores transfer to deployment. Arp et al. [2] catalog recurring pitfalls in security ML, and Lanvin et al. [13] document labeling and duplication errors in CICIDS2017 [24] that measurably inflate reported performance; Sarhan et al. [22] similarly show that the choice of standard feature set shapes both generalizability and explainability. Our companion ML study responds by selecting a leakage-free SHAP Top-30 model [15], and the present paper deploys exactly that model, so the operational numbers reported here do not belong to a configuration that would fail a stricter offline audit. Concept drift [7] lies outside the present scope and motivates the retraining direction we outline in the conclusion. Table 1 positions our system against representative related work along the five axes above: multi-node role separation, streaming ingest, joint energy and latency reporting, leakage-aware model selection, and open code.

3 System Architecture

3.1 Split Deployment Overview

Fig. 1 shows the split-deployment architecture. A host PC runs Kafka, PostgreSQL, InfluxDB, and Grafana. Two Jetson Orin Nano Super nodes consume messages from the

[6] `https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/nano-super-developer-kit/`

Table 1. Capability comparison with prior edge IDS deployments.

Reference	Platform	Multi-node roles	Streaming ingest	Energy & p95 latency	Leakage-aware model	Open code
Sforzin et al. [23]	Raspberry Pi	No	No	—	—	—
López-Martín et al. [14]	GPU server	No	No	—	—	—
Nguyen et al. [20]	IoT gateways	Federated	No	—	—	—
Hasan et al. [9]	$1\times$ Jetson Nano	No	No	—	—	—
Chithra Rani et al. [4]	$1\times$ Jetson Nano	No	No	—	—	—
Nasir et al. [18]	Edge-like (simulated)	No	No	—	—	—
The proposed system	**$2\times$ Jetson Orin Nano S.**	**Yes**	**Yes (Kafka)**	**Yes**	**Yes**	**Yes**

"—": not reported or not applicable.

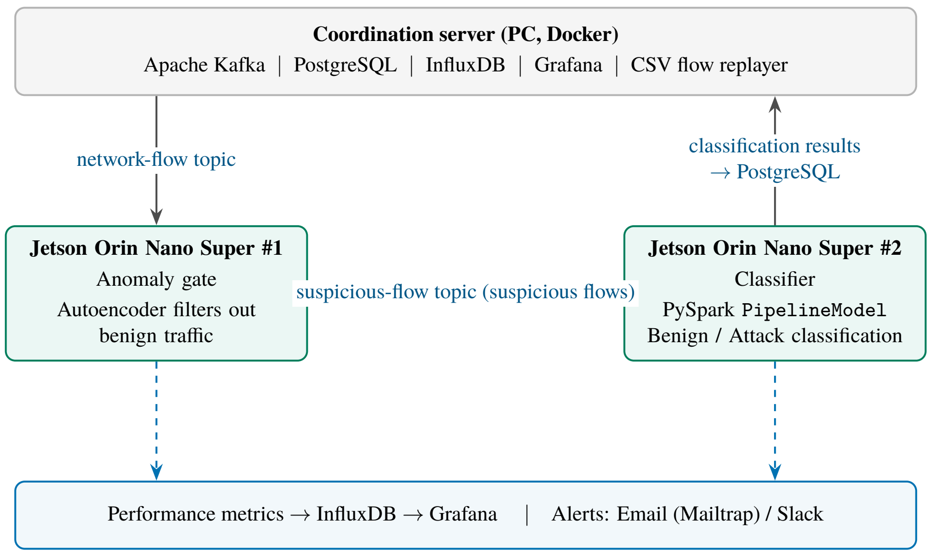

Fig. 1. Split-deployment architecture (PC + $2\times$ Jetson Orin Nano Super).

network-flow topic. In pipeline-split mode, Jetson #1 runs the anomaly gate and forwards only suspicious flows to a dedicated suspicious-flow topic. Jetson #2 then applies the PySpark classifier and writes the classification results back to PostgreSQL. Both nodes stream performance metrics to InfluxDB/Grafana and raise alerts via Email/Slack (dashed arrows in Fig. 1).

3.2 Pipeline Roles

- **Anomaly gate:** a lightweight autoencoder + scaler; the MSE threshold is calibrated offline as a quantile of a held-out benign validation split (10% of benign training flows, never the test set), so the operating point is committed before evaluation.
- **Classifier:** PySpark `PipelineModel` with the leakage-free SHAP-selected feature set.
- **Combined role:** gate and classifier in one process, used for horizontal scaling via Kafka consumer groups.

Fig. 2. Real-time Grafana monitoring of the edge IDS during CICIDS2017 replay.

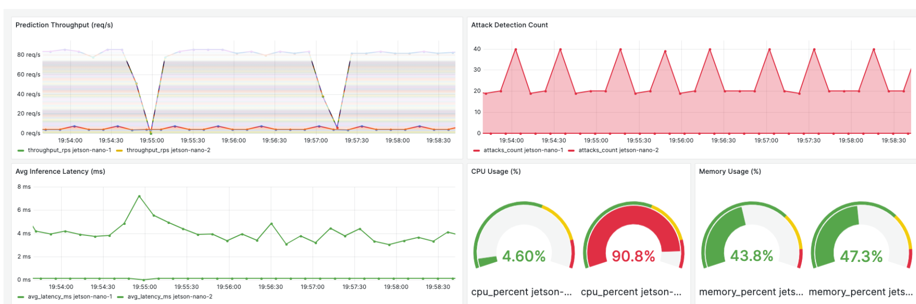

Fig. 3. Grafana dashboard at peak load in the split deployment, showing the load asymmetry between the two pipeline roles.

3.3 Monitoring and Alerting

Each node tags its metrics with a unique node identifier in InfluxDB and PostgreSQL. Alerts are emitted from a designated primary node only, avoiding duplicate notifications. Fig. 2 shows the live Grafana dashboard while CICIDS2017 flows are replayed through the split deployment: prediction throughput, detected-attack count, inference latency, and per-node CPU/RAM utilization are all tracked in real time, alongside the most recent attack alerts persisted to PostgreSQL. Fig. 3 captures the dashboard at peak load: the gate node idles at 4.6% CPU while the PySpark classifier node reaches 90.8%, so the classifier dominates the compute budget.

4 Implementation

Training and model export run on a Spark standalone cluster (host PC as master, two Jetson Orin Nano Super devices as workers) using Apache Spark [27] and its MLlib library [16]; the cluster topology is shown in Fig. 4. The host only coordinates (Spark master) and does not run an executor, so all reported training compute reflects the two homogeneous Jetson workers rather than the stronger host CPU. The edge stack is implemented in Python with configurable per-node roles. Every offline run logs its configuration (random seed, leakage/dedup switches, library versions) so each exported model is traceable to the exact pipeline that produced it.

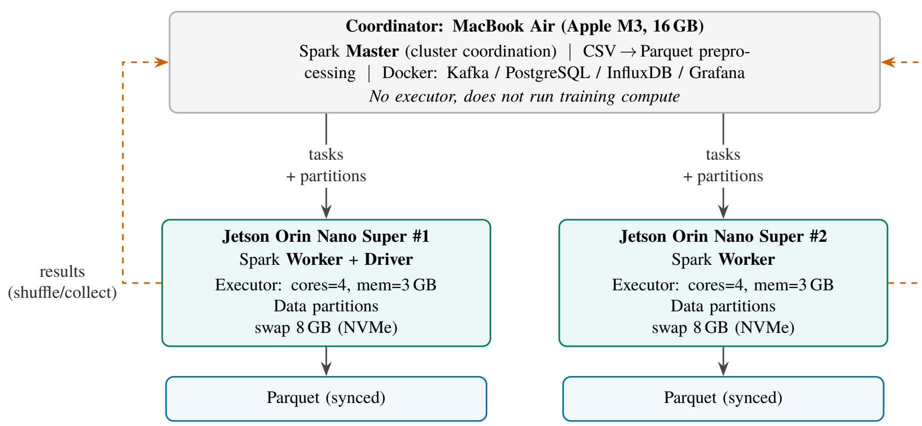

Fig. 4. Apache Spark standalone training cluster: the host as *master* (no executor), two Jetson Orin Nano Super devices as *worker/executor* nodes.

5 From Spark Training to Edge Deployment

This paper does not compare Spark against Jetson Orin Nano Super as competing platforms. The two components address complementary phases of the same train–deploy pipeline (Fig. 5). Spark answers which model and how many features achieve acceptable accuracy on the full CICIDS2017 dataset [24]; the Jetson Orin Nano Super answers whether that model can then run in real time under memory, thermal, and bandwidth constraints at the network edge [25].

5.1 Offline Phase (Spark ML), Leakage-Aware

In our companion ML study, we evaluate four feature configurations (baseline with all features, RF Importance Top-30, SHAP Top-30, and PCA with k=40) under a fixed 80/20 train–test split, using eight Spark ML classifiers and statistical validation (repeated

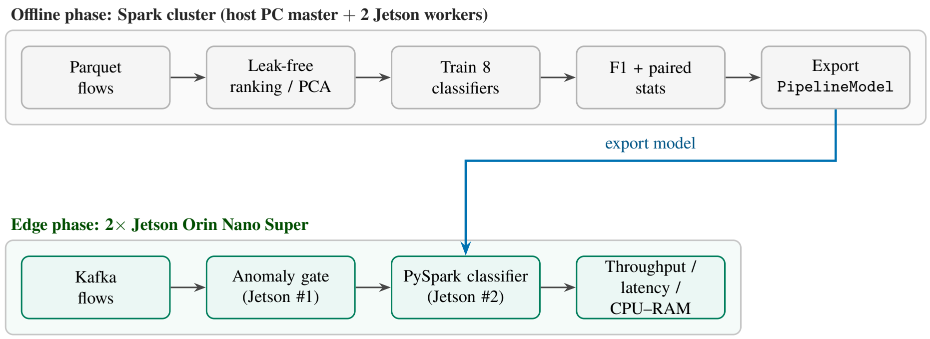

Fig. 5. Two-phase train–deploy pipeline linking Spark ML experiments to Jetson edge inference.

paired resampling and permutation tests). PCA uses k=40 rather than 30 because it extracts components (linear combinations spreading variance across many axes) instead of selecting original features, so it needs more components to retain comparable variance; across the common $\{20, 30, 40\}$ sweep, k=40 retains the most variance and is the fairest representative operating point for PCA. LightGBM is excluded because its SynapseML native libraries are x86_64-only and cannot run on the ARM64 Jetson Orin Nano Super deployment target. The offline protocol is leakage-aware [2]: it removes information that would be unavailable at deployment time but that otherwise inflates test scores. Motivated by documented labeling and duplication errors in CICIDS2017 [13], the label-leaking `destination_port` feature is removed and flows that are identical on all feature columns are de-duplicated before splitting. This yields realistic F1 values (Table 2) rather than the near-perfect scores obtained when these leaks are left in place. Offline metrics constrain which exported model is worth deploying; they are not directly comparable to edge throughput or latency.

Table 2. Bridge table: offline Spark selection criteria from the leakage-aware companion study.

Configuration	F1	Features	Size (MB)	Edge rationale
Baseline + XGBoost	0.9971	77	2.32	Reference accuracy
SHAP Top-30 + XGBoost	0.9970	30	0.003†	Explainable; fewer features (deployed feature set)
RF Top-30 + XGBoost	0.9922	30	2.52	Same feature count; embedded ranking
PCA k=40 + XGBoost	0.9939	40	3.82	Unsupervised reduction

†Serialized on the training cluster, where the distributed model-saving step scatters the tree-data partitions across executors; KB-scale entries captured only driver-local metadata and under-report the true size (a 30-feature XGBoost is in reality ≈2.5 MB, on par with RF Top-30, since ensemble size depends on trees/nodes rather than input features; with fewer split candidates trees may even need slightly more nodes than the baseline, cf. 2.52 vs. 2.32 MB). The single-node export used for deployment measures sizes reliably (cf. Table 5).

5.2 Edge Phase (Jetson Orin Nano Super)

The exported `PipelineModel` (VectorAssembler, StandardScaler, classifier) is loaded on Jetson #2 with only the selected SHAP features, reducing assembly and inference cost relative to the 77-feature baseline. The deployed input vector contains 29 features: the SHAP Top-30 ranking includes `destination_port`, which is excluded from the exported model as a label-leaking channel on CICIDS2017, consistent with the leakage-aware offline protocol. A lightweight autoencoder gate on Jetson #1 filters benign flows before they reach Spark inference, further lowering load on the classifier node. End-to-end edge evaluation measures throughput (flows/s), latency p50/p95 (ms), per-node CPU/RAM/temperature, attack-class F1, and gate-skip ratio, none of which the offline Spark experiments capture.

5.3 Division of Evaluation Metrics

Table 3 clarifies which metric belongs to which phase. Mixing offline F1 with edge latency in a single ranking would conflate accuracy with operational feasibility.

Table 3. Metric split between offline Spark training and edge Jetson inference.

Aspect	Spark (offline)	Jetson (edge)
Primary goal	Classification accuracy	Real-time feasibility
Main metrics	F1, AUC-PR, Precision/Recall	Throughput, latency p50/p95
Secondary metrics	Train time, batch pred. time, model size	CPU, RAM, temperature, gate skip %
Data	CICIDS2017 train/test split	Kafka replay at 100 flows/s
Scale	Millions of flow records	2 nodes, 8 GB RAM each

We deploy the SHAP Top-30 feature set on Jetson #2 with a Random Forest classifier (offline F1 0.9955, within 0.002 of the best XGBoost score in Table 2): it retains near-baseline F1 with substantially fewer features, and Random Forest is a native Spark ML estimator, so the exported `PipelineModel` loads on the ARM board without the additional native libraries that Spark's XGBoost integration requires. Sect. 7 then evaluates this choice under streaming load; the companion ML paper reports the full four-method comparison and drift analysis.

6 Experimental Setup

The testbed comprises two NVIDIA Jetson Orin Nano Super Developer Kits and a host PC running Docker Compose, connected over a Wi-Fi LAN. Each Jetson board pairs a 6-core Arm Cortex-A78AE CPU with an NVIDIA Ampere GPU featuring 1024 CUDA and 32 Tensor cores, delivering roughly 67 TOPS, and is equipped with 8 GB of LPDDR5 memory and a 256 GB NVMe SSD. On the software side, streaming inference runs on PySpark 3.4.1 over Apache Kafka 3.5 (Confluent Platform 7.5.0), while InfluxDB and Grafana collect and visualize runtime metrics. The workload is a CSV replay of CICIDS2017 at 100 flows/s, against which we compare a single-node full pipeline with the three distributed modes A (pipeline split), B (horizontal Kafka scaling), and C (Spark standalone cluster). For every run we report throughput, p50/p95 latency (per-node percentiles over raw per-flow samples; the cluster p95 is the worst node's p95, plus end-to-end send-to-verdict latency from producer timestamps under NTP-synchronized clocks), per-node CPU/RAM/temperature, attack-class F1, gate-skip ratio, and energy-per-inference (mJ, via `tegrastats`, idle baseline subtracted).

7 Results

We run each deployment mode through a host-side orchestration script that starts the corresponding pipeline roles on the Jetson nodes over SSH, drives the load, and collects per-node logs. Each configuration is measured over at least three repetitions preceded by representative warmup traffic (excluded from the measured window, so JVM/JIT warmup does not inflate early-batch latency); Table 4 reports mean $\pm$ std across repetitions. All metric queries are aligned to the exact load window rather than a trailing time range. Latency percentiles are computed per node over the raw per-flow samples logged during the window, and pipeline throughput is counted as final verdicts per second (a flow's verdict is issued either at the gate, for skipped flows, or at the classifier), which avoids double-counting forwarded flows in pipeline-split mode. Aggregated results are reported in Table 4, with throughput and p95 latency across modes visualized in Fig. 6.

Table 4. Distributed deployment comparison on the 2-node Jetson cluster (mean $\pm$ std over repetitions).

Mode	Throughput (flows/s)	Latency p95 (ms)	Gate skip (%)	Energy/inf. (mJ)	Attack F1
Single node	2.60 ± 0.41	12.3 ± 3.0	(inline)	2490	†
A: Pipeline split	62.5 ± 0.6	13.1 ± 2.6	95.8	178	†
B: Horizontal	5.9 ± 1.4	21.6 ± 19.2	(inline)	2250	†
C: Spark cluster	90.0 ± 25.6	23.9 ± 17.8	97.9	119	†

Latency p95 is computed from the raw per-flow logs and reported for the *worst* node. "(inline)": in the combined role the gate runs in-process and its skips are persisted as verdicts, so no separate skip ratio applies. Energy is *total-board* energy per verdict: at this load the idle-subtracted (active) power delta was below the `tegrastats` noise floor ($<$0.15 W), so the figure is dominated by board idle power and measures how effectively each mode amortizes it over verdicts. † Detection quality is mode-independent (identical exported models); offline attack-class F1 is reported in Table 2.

Energy efficiency. On-board power is sampled with `tegrastats` during each run (30 s idle baseline, then the load window; for two-node modes the powers of both nodes are summed). At 100 flows/s the measured active (idle-subtracted) power delta was below the sampling noise floor ($<$0.15 W on every node), so Table 4 reports total-board energy per verdict instead, that is, board power amortized over verdict throughput. The modes still separate clearly: the gate-based modes (A and C) amortize the two boards' power over 24–35$\times$ more verdicts than the single node, cutting energy per verdict from

$\approx$2.5 J to 0.12–0.18 J. Even so, the figures should be read as a deployment-level rather than a model-level energy metric.

Inference-engine baseline (cost of the unified stack). We deploy a Spark `PipelineModel` on the edge for consistency with the distributed training phase: the same stack trains on the cluster and serves on the device. To quantify the price of that unification, we benchmark the same RandomForest (matched hyper-parameters) on the same deployed SHAP-selected feature set (Sect. 5) through lighter single-node engines: scikit-learn (CPU, no JVM) and ONNX Runtime (CPU execution provider), in Table 5. Accuracy is comparable by construction; the comparison targets inference cost. All three engines are measured on the classifier node under identical board conditions at batch size 10; energy is active (idle-subtracted) per inference at engine saturation. The JVM/Spark stack does incur substantially higher inference cost than scikit-learn on the 8 GB ARM board (9$\times$ lower throughput, $\approx$11$\times$ higher p95, roughly double the RAM share, and $\approx$25$\times$ the active energy per inference), and compiling the same forest to ONNX Runtime widens the gap much further: $\approx$7,900 flows/s at a p95 of 1.1 ms, i.e. 348$\times$ the deployed Spark throughput at $\approx$2% of the per-inference energy of scikit-learn. That gap motivates exporting the `PipelineModel` to ONNX/TensorRT for inference while retaining Spark for training; wiring the ONNX session into the streaming pipeline (and exploiting the GPU via TensorRT) is left as future work.

Table 5. Single-node inference cost of the same model and feature set across engines.

Engine	Throughput (flows/s)	p95 latency (ms)	RAM (%)	Energy/inf. (mJ)
PySpark (deployed)	22.6	532	24.3	71.6
scikit-learn	204.4	49.8	13.3	2.91
ONNX Runtime	7870.8	1.1	15.7	0.06

The anomaly gate is tunable rather than a single fixed point: sweeping its threshold trades attack recall against the fraction of benign flows offloaded from Spark (Fig. 7). An operator can therefore pick an operating point that fits the available edge budget.

8 Discussion

Pipeline split relieves Spark of most benign traffic, and the realized benefit scales with the gate-skip ratio ($\approx$96% in our runs): decoupling the gate from the Spark classifier lifted verdict throughput from 2.6 to 62.5 flows/s (24$\times$) at a nearly unchanged p95. Horizontal scaling of the full role roughly doubles the single node (5.9 vs. 2.6 flows/s) but remains Spark-bound, because every node still runs the classifier in-process. The Spark standalone cluster variant reached the highest mean throughput (90 flows/s) but with large run-to-run variance ($\pm$26) and an extra host dependency (the Spark master), so pipeline split remains the recommended default; cluster mode is most useful when model or batch size exceeds single-node memory.

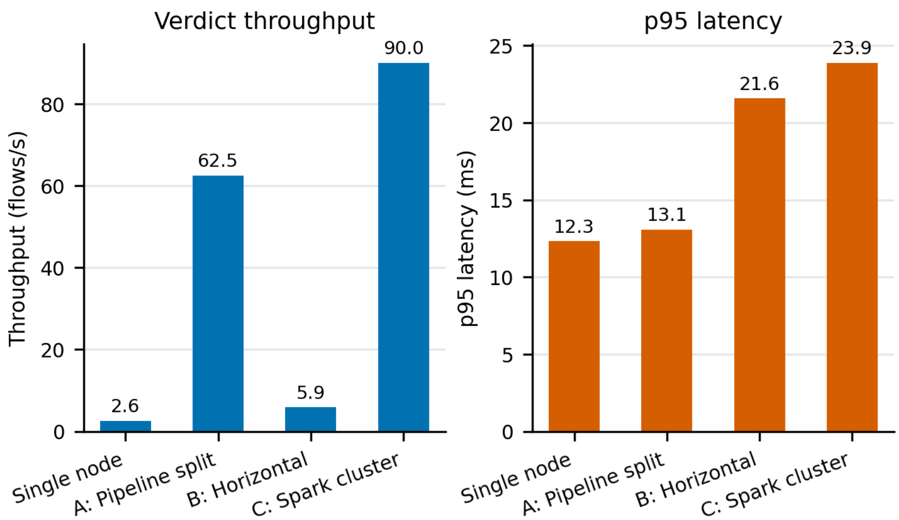

Fig. 6. Deployment-mode comparison: verdict throughput (left) and p95 latency (right).

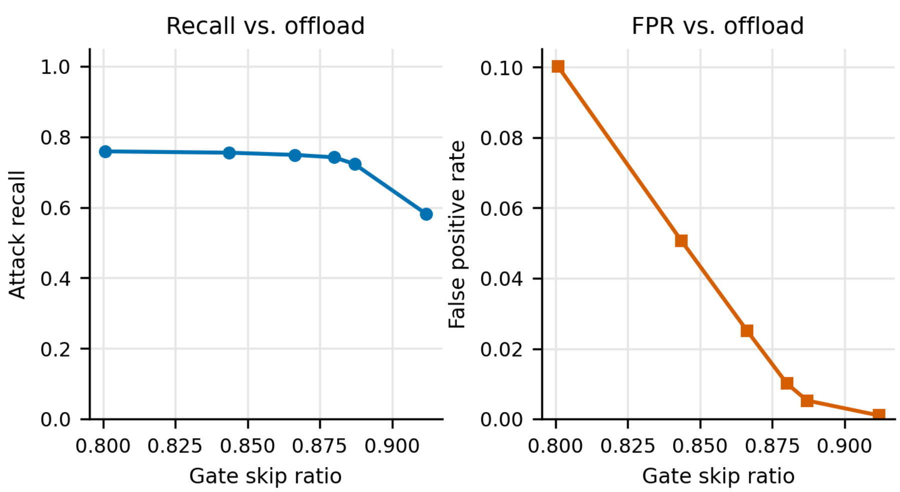

Fig. 7. Anomaly-gate operating curve: attack recall (left) and false-positive rate (right) vs. gate-skip ratio.

As summarized in Sect. 5, offline Spark experiments and edge Jetson benchmarks answer different questions. Feature reduction from 77 to 30 dimensions (SHAP Top-30) directly reduces `VectorAssembler` and `PipelineModel` cost on the Orin Nano Super, while the anomaly gate skips a large fraction of benign flows before Spark inference is invoked. These two choices connect the accuracy-oriented conclusions of the companion ML study to a deployable, measurable edge architecture.

Limitations. Benchmarks are obtained on a two-node lab cluster over Wi-Fi; wired networking and larger clusters may shift latency tails. The anomaly-gate threshold trades attack recall against the gate-skip ratio, so we report it as an operating curve (Fig. 7) and leave live-traffic re-calibration to future work. Each configuration is repeated at least three times with warmup excluded (Sect. 7); run-to-run variance on a thermally constrained board is reported alongside the means. Latency percentiles are computed per node from raw per-flow samples, and the cluster-level p95 is conservatively the worst node's p95; end-to-end latency (from producer send to final verdict) relies on NTP-synchronized clocks and excludes DB-write cost. Running PySpark on the JVM on an 8 GB Jetson incurs substantial overhead: Spark is chosen for consistency with the distributed training phase, the overhead is quantified in Table 5, and the $\sim$67-TOPS GPU remains unexploited by the CPU-only pipeline (TensorRT is future work). The energy-per-inference figures subtract a single pre-run idle baseline, so within-run drift in background power is not captured; in split mode they sum both nodes' active energy, and the gate-skip saving does not subtract the autoencoder cost incurred on all flows. Finally, results are tied to CICIDS2017 replay: live traffic and deliberately evasive (adversarial) traffic remain untested.

9 Conclusion

We presented a reproducible distributed edge IDS for Jetson Orin Nano Super clusters that pairs Kafka streaming with PySpark inference and deploys a model validated under a leakage-aware offline protocol, closing the gap between an accuracy-oriented offline study and a measurable edge deployment. Splitting the pipeline so that a lightweight autoencoder gate and the Spark classifier occupy separate boards raises verdict throughput from 2.6 to 62.5 flows/s ($24\times$) at a p95 of $\approx$13 ms that is essentially unchanged, because the gate absorbs 95.8% of benign traffic before it reaches the classifier; amortized over that many more verdicts, board energy falls from $\approx$2.5 J to 0.18 J per verdict. Horizontal scaling of the full role stays Spark-bound (5.9 flows/s), while the Spark standalone cluster attains the highest mean throughput (90 flows/s) at a far larger run-to-run spread ($\pm$25.6 vs. $\pm$0.6 flows/s), so pipeline split remains our recommended default and cluster mode is reserved for models that exceed single-node memory. We also quantified the price of running one stack for both training and serving: on the same forest and the same feature set, ONNX Runtime sustains $348\times$ the throughput of the deployed PySpark path, a concrete target for the next iteration of the serving layer. Future work includes TensorRT optimization, live-traffic validation, and federated retraining under drift [10].

Disclosure of Interests. The authors declare that they have no competing interests.

References

1. Alrawashdeh, K., Purdy, C.: Toward an online anomaly intrusion detection system based on deep learning. In: Proc. 15th IEEE Int. Conf. on Machine Learning and Applications (ICMLA). pp. 195–200 (2016). https://doi.org/10.1109/ICMLA.2016.0040

2. Arp, D., Quiring, E., Pendlebury, F., Warnecke, A., Pierazzi, F., Wressnegger, C., Cavallaro, L., Rieck, K.: Dos and don'ts of machine learning in computer security. In: 31st USENIX Security Symposium (USENIX Security 22). pp. 3971–3988 (2022)
3. Chandola, V., Banerjee, A., Kumar, V.: Anomaly detection: A survey. ACM Computing Surveys **41**(3), 1–58 (2009). https://doi.org/10.1145/1541880.1541882
4. Chithra Rani, P.R., Baalaji, K.: Deep learning-based ensemble stacking for enhanced intrusion detection in IoT-edge platforms. Discover Applied Sciences **7**, 928 (2025). https://doi.org/10.1007/s42452-025-06871-z
5. Diaz-Gorrin, J., Caballero-Gil, C., Brankovic, L.: Enhancing network security with generative AI on Jetson Orin Nano. Applied Sciences **16**(3), 1442 (2026). https://doi.org/10.3390/app16031442
6. Ferrag, M.A., Maglaras, L., Janicke, H., Jiang, J., Shu, L.: Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study. Journal of Information Security and Applications **50**, 102419 (2020)
7. Gama, J., Zliobaitė, I., Bifet, A., Pechenizkiy, M., Bouchachia, A.: A survey on concept drift adaptation. ACM Computing Surveys **46**(4), 1–37 (2014)
8. Hamdan, S., Ayyash, M., Almajali, S.: Edge-computing architectures for internet of things applications: A survey. Sensors **20**(22), 6441 (2020). https://doi.org/10.3390/s20226441
9. Hasan, T., Hossain, A., Ansari, M.Q., Syed, T.H.: Enhanced intrusion detection in IIoT networks: A lightweight approach with autoencoder-based feature learning (2025), arXiv:2501.15266
10. Hernandez-Ramos, J.L., Karopoulos, G., Chatzoglou, E., Kouliaridis, V., Marmol, E., Gonzalez-Vidal, A., Kambourakis, G.: Intrusion detection based on federated learning: A systematic review. ACM Computing Surveys **57**(12), 1–65 (2025). https://doi.org/10.1145/3731596
11. Khraisat, A., Gondal, I., Vamplew, P., Kamruzzaman, J.: Survey of intrusion detection systems: techniques, datasets and challenges. Cybersecurity **2**(1), 1–22 (2019)
12. Kreps, J., Narkhede, N., Rao, J.: Kafka: a distributed messaging system for log processing. In: Proceedings of the NetDB Workshop (2011)
13. Lanvin, M., Gimenez, P.F., Han, Y., Majorczyk, F., Mé, L., Totel, É.: Errors in the CICIDS2017 dataset and the significant differences in detection performances it makes. In: Risks and Security of Internet and Systems (CRiSIS 2022), Revised Selected Papers. pp. 18–33. Lecture Notes in Computer Science, Springer (2023). https://doi.org/10.1007/978-3-031-31108-6_2
14. Lopez-Martin, M., Carro, B., Sanchez-Esguevillas, A., Lloret, J.: Conditional variational autoencoder for prediction and feature recovery applied to intrusion detection in iot. Sensors **17**(9), 1967 (2017). https://doi.org/10.3390/s17091967
15. Lundberg, S.M., Lee, S.I.: A unified approach to interpreting model predictions. Advances in Neural Information Processing Systems **30** (2017)
16. Meng, X., Bradley, J., Yavuz, B., Sparks, E., Venkataraman, S., Liu, D., Freeman, J., Tsai, D., Amde, M., Owen, S., Xin, D., Xin, R., Franklin, M.J., Zadeh, R., Zaharia, M., Talwalkar, A.: MLlib: Machine learning in Apache Spark. Journal of Machine Learning Research **17**(34), 1–7 (2016)
17. Mohan Kiran Kumar, K., Sreekar, M., Ullas, K., Supriya, M.: Distributed intrusion detection system using Kafka and Spark streaming. In: 2025 International Conference on Visual Analytics and Data Visualization (ICVADV). pp. 302–309. IEEE (2025). https://doi.org/10.1109/ICVADV63329.2025.10961691
18. Nasir, K., Badri, S.K., Alghazzawi, D.M., Alghamdi, M.Y., Alkhozae, M., Almakky, A., Alhazmi, R.M., Asghar, M.Z.: HED-ID: an edge-deployable and explainable intrusion detection system optimized via metaheuristic learning. Scientific Reports **16**, 2313 (2026). https://doi.org/10.1038/s41598-025-32183-8

19. Nassef, L., Alghamdi, M.I., Ben Chaabane, S., Abbas, Q., Alawad, W.M., Albalawi, O.H., Alqaisi, O.I., Fakieh, B.: Lightweight and energy-aware intrusion detection for industrial IoT using TinyML and edge AI. Scientific Reports **16**(1), 18524 (2026). https://doi.org/10.1038/s41598-026-50690-0
20. Nguyen, T.D., Marchal, S., Miettinen, M., Fereidooni, H., Asokan, N., Sadeghi, A.R.: DÏoT: A federated self-learning anomaly detection system for IoT. In: Proc. IEEE 39th Int. Conf. on Distributed Computing Systems (ICDCS). pp. 756–767 (2019). https://doi.org/10.1109/ICDCS.2019.00080
21. Ruff, L., Kauffmann, J.R., Vandermeulen, R.A., Montavon, G., Samek, W., Kloft, M., Dietterich, T.G., Müller, K.R.: A unifying review of deep and shallow anomaly detection. Proceedings of the IEEE **109**(5), 756–795 (2021)
22. Sarhan, M., Layeghy, S., Portmann, M.: Evaluating standard feature sets towards increased generalisability and explainability of ml-based network intrusion detection. Big Data Research **30**, 100359 (2022). https://doi.org/10.1016/j.bdr.2022.100359
23. Sforzin, A., Gómez Mármol, F., Conti, M., Bohli, J.M.: RPiDS: Raspberry pi IDS — a fruitful intrusion detection system for IoT. In: Proc. Int. IEEE Conf. on Ubiquitous Intelligence & Computing (UIC/ATC/ScalCom). pp. 440–448 (2016). https://doi.org/10.1109/UIC-ATC-ScalCom-CBDCom-IoP-SmartWorld.2016.0080
24. Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A.: Toward generating a new intrusion detection dataset and intrusion traffic characterization. In: Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP). pp. 108–116 (2018)
25. Shi, W., Cao, J., Zhang, Q., Li, Y., Xu, L.: Edge computing: Vision and challenges. IEEE Internet of Things Journal **3**(5), 637–646 (2016). https://doi.org/10.1109/JIOT.2016.2579198
26. Vinayakumar, R., Alazab, M., Soman, K.P., Poornachandran, P., Al-Nemrat, A., Venkatraman, S.: Deep learning approach for intelligent intrusion detection system. IEEE Access **7**, 41525–41550 (2019)
27. Zaharia, M., Chowdhury, M., Franklin, M.J., Shenker, S., Stoica, I.: Spark: Cluster computing with working sets. In: Proceedings of the 2nd USENIX Workshop on Hot Topics in Cloud Computing (HotCloud'10) (2010)
28. Zhang, Z., Zhang, L., Xu, J., Chen, Z., Wang, P.: An edge-deployable lightweight intrusion detection system for industrial control. Electronics **15**(3), 644 (2026). https://doi.org/10.3390/electronics15030644